

String handling

Revise 360 lesson plan — 60 minutes

Unit	Beyond OCR J277: a Python pathway supporting 2.1, 2.2 and 2.3
Lesson	String handling (lesson 6 of the unit)
Lesson ID	pr-106
Experience ID	pr-106
Plan ID	PLAN-PR-L06
Content version	8ed7d17fb3a1
Website	https://revise360.co.uk/experience.html?id=pr-106

Big question, outcomes and success criteria

Big question — Why are the characters of a string numbered from 0 rather than from 1?

Prerequisites

The lesson before this one is pr-105. Pupils should already have finished it: the starter below retrieves from it.

Earlier in the unit: pr-101, pr-102, pr-103, pr-104.

Assessable outcomes

- pr-106-o1** — know how to find the length of a string and the character at a given position
- pr-106-o2** — understand how a slice takes part of a string, and why its end position is left out
- pr-106-o3** — know how to change case, search inside text, and move between a character and its code

Success criteria the pupil can check against

Taken from the briefs the independent activities themselves show, so these are the words the pupil sees. Their work has met the outcome when it does these things.

- The user types in one word. **pr-106-s1-a6**
- Three lines of output, in that order. **pr-106-s1-a6**
- Display exactly one character. **pr-106-s1-a7**
- The user types in two lines: one word, then a whole number. **pr-106-s1-a8**
- Display exactly: no such position **pr-106-s1-a8**
- One line of output. **pr-106-s1-a8**
- The user types in one word of at least two characters. **pr-106-s2-a5**
- The user types in two lines: a line of text, then a whole number. **pr-106-s2-a6**

Vocabulary, resources and preparation

Key vocabulary — Index, Slice, Character set, ord() and chr()

Resources

SLIDES	presentations/PR_L06_StringHandling_Lesson.pptx
WORKSHEET	worksheets/PR_L06_StringHandling_Worksheet.docx
ONLINE EXPERIENCE	experience.html?id=pr-106

Before the lesson

1. Sign in on the **teacher dashboard** (`teacher.html`) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the Python runtime and the 360 image are already cached.
4. Check headsets or devices are charged and on the school network.

How pupils get in

1. Go to `topics.html` on `revise360.co.uk`.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

Worth knowing before you promise anything. In this build `APP_CONFIG.backendUrl` is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
0-5	Retrieve	Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites.	Answer the starter on the worksheet.	The starter box on <code>PR_L06_StringHandling_Worksheet.docx</code> is filled in.
5-8	Review the starter	Take answers; correct the one most got wrong before moving on.	Correct their own starter.	A correction written, not just a tick.
8-16	Explain and model	The new idea, then the worked example, typed live rather than shown finished.	Watch the example being built, and say what each line does.	They can name the line that does the job, not just copy it.
16-19	Set up and settle	Sign-in, launch, and what to record on paper before anyone types.	Sign in, open the lesson, read the task before typing.	Everyone is at station 1 activity 1, not typing yet.
19-48	Online: work the stations in order	Pupils work; you circulate and read the editor over shoulders.	Work the stations in order, running and checking as they go.	The station badges fill in; the editor shows their own program, not the starter.
48-53	Check understanding	The hinge question, hands down, then the reteach if the class splits.	Answer the hinge question, hands down.	The split across the options, before any reteach.
53-58	Apply or extend	The challenge activity, or the exam practice on the worksheet.	The Challenge activity, or the exam practice.	worksheet, Exam practice a) to c)
58-60	Exit and set up the next lesson	The exit question and what to finish before next time.	Write the exit answer.	One sentence that answers the big question.

The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

1. Length and position

- `len()` gives the number of characters in a string, spaces included.
- Characters are numbered from 0, so the last one sits at `len() - 1`.
- Common mistake: asking for position `len()`, which is one past the end and stops the program.

Challenge on this wall: Which position holds the last character of a six-letter word?

2. Taking part of a string

- A slice takes a run of characters, so `word[0:3]` gives the first three.
- The first number is included and the second is not, exactly like `range()`.
- Common mistake: expecting `word[0:3]` to give four characters rather than three.

Challenge on this wall: Which slice gives everything except the first and last characters?

3. Upper and lower case

- `upper()` and `lower()` hand back a new string and leave the original untouched.
- Comparing two strings in the same case ignores how they happened to be typed.
- Common mistake: calling `lower()` but never storing or using what it gives back.

Challenge on this wall: Why does a program put two words into the same case before deciding they match?

4. Searching inside text

- in tests whether one string appears anywhere inside another.
- `find()` gives the position it starts at, or -1 when it is not there at all.
- Common mistake: treating `find()`'s 0 as 'not found', when 0 means the very start.

Challenge on this wall: How can a program tell 'not found' apart from 'found at the start'?

5. Character codes

- Every character is stored as a number, and `ord()` gives that number.
- `chr()` goes the other way, turning a code back into its character.
- Common mistake: adding 1 to the letter itself, when it is the code that has to be added to.

Challenge on this wall: How would you work out the letter that comes after a given one?

6. Putting it together

- Build a new string one character at a time inside a loop over the original.
- `isalpha()` and `isdigit()` sort the letters from the digits as you go.
- Common mistake: trying to change one character in place, when a string cannot be edited.

Challenge on this wall: How would you build a reversed copy of a word one character at a time?

The example to model

A string is a piece of text. `len()` counts how many characters it holds and hands back a whole number. Spaces count as characters too.

```
w = "otter"
print(len(w))
```

It displays 5

Decisions to model out loud, not just the finished answer: say why you chose that name, why that line comes before the next one, and what you would try first if it did not work. The example deliberately uses different data from the task, so a pupil cannot copy it and be finished.

Retrieval and the check for understanding

Starter — The variable `word` holds `Cavendish`. Write down what each of these gives: `len(word)`, `word[0]`, `word[0:4]`, `word[-1]` and `word.upper()`.

Answer space: 4 lines on the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (pr-106-s4-c2), so the wording a pupil sees matches the wording you say.

What will this program display?

Correct: 4 -1

What a pupil might choose instead

- 5 -1
- 4 0

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

The course's own response to a wrong answer here: wet and the space fill positions 0 to 3, so paint starts at 4. A word that is not there gives -1.

One response, not four. one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

Misconceptions this lesson addresses

- asking for position `len()`, which is one past the end and stops the program. [wall panel](#)
- The positions stop one short of the length. Which position does `len(w)` point at? [pr-106-s1-a7, a Fix it activity](#)
- expecting `word[0:3]` to give four characters rather than three. [wall panel](#)
- Look at the data type of the value being used inside the square brackets. [pr-106-s2-a6, a Fix it activity](#)
- calling `lower()` but never storing or using what it gives back. [wall panel](#)
- `lower()` hands a new string back. Is anything being done with those two answers? [pr-106-s3-a6, a Fix it activity](#)
- treating `find()`'s 0 as 'not found', when 0 means the very start. [wall panel](#)
- Which answers from `find()` really mean the word was not there? [pr-106-s4-a6, a Fix it activity](#)

Guided to independent practice

STAGE	ACTIVITIES	WHAT IT IS
Guided	26	Support is in the activity: an example to run, an output to predict, one thing to change, a gap to fill.
Independent	12	The pupil writes or fixes it with the brief and nothing else.
Challenge	6	The hardest activity on each station. It is not optional and it is not extra: it is the last step of the station.

Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

Delivering the online experience

Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

The stations, and what to record at each

#	STATION	ACTIVITIES	RECORD ON PAPER
1	Length and position	8	A negative index counts from the end, so word[-1] is always the...
2	Taking part of a string	7	In a slice the first number is included and the second one is...
3	Upper and lower case	7	Strings are compared by character code, so in Python every capital sorts before every...
4	Searching inside text	7	Two methods answer the commonest search questions on their own: startswith() and...
5	Character codes	7	A is 65 and a is 97, so the gap between a capital and its small letter is always...
6	Putting it together	7	A word that reads the same backwards as forwards is called a...
final	Final challenge: what does it give?	1	the answer to the station's challenge

Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

Python: the order is not negotiable. Each activity teaches what the next one needs, so a pupil works through them in order and completes every one. A shorter lesson, homework or a slow worker does not authorise skipping: the pupil stops where they are and resumes at their earliest unfinished question next time, because progress saves after every answer.

A paper activity, a help request or a tick on this plan does not mark an online question complete, and there is no teacher bypass. If a pupil is stuck after the hint, that is the moment to teach them, not to move them on.

Assessment, evidence and unfinished work

ID	QUESTION	MARKS	WHERE
pr-106-x1	A program tries to print the last character of a word with <code>print(word[len(word)])</code> and stops with an error. Explain why it fails, and write the line corrected.	3	worksheet, Exam practice a); deck, Exam practice slide
pr-106-x2	Write a program that reads a line of text and prints how many of its characters are capital letters.	4	worksheet, Exam practice b); deck, Exam practice slide
pr-106-x3	Write a program that reads a word and prints palindrome when it reads the same both ways once capitals are ignored, and not a palindrome when it does not.	5	worksheet, Exam practice c); deck, Exam practice slide

Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/pr-106.json`, which is not published with the website.

Every programming activity in this lesson is marked by running the pupil's program against its test cases, so the mark is the program's behaviour and not its appearance. The model solutions are in `answers/codebank/pr-106.json`.

Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What will this program display?** — a t; c c
- **What will this program display?** — 3 a; 4 t
- **What will this program display?** — sto ston; st to
- **What will this program display?** — True True; False False
- **What will this program display?** — 5 -1; 4 0

Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

Exit, next lesson and homework

Exit check

One of these, written, not a show of hands:

- Using `len()` and an index on a string
- Taking a slice out of a string
- Searching text and using `ord()` and `chr()`

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

Retrieval link to next lesson

Next is pr-107, Lists. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

Homework

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

Shorter lessons, longer lessons and no internet

If this lesson takes more than an hour

It will. There are 44 activities across 6 stations, and a beginner does not finish 44 of them in twenty-nine minutes. Plan it as two or three sessions of an hour: stations 1 and 2 in the first, 3 and 4 in the second, 5, 6 and the final challenge in the

third. Each session keeps the same starter-explain-model opening; the online block resumes where the class got to. Honest total: two to three hours.

A 40 to 45 minute lesson

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

A shorter lesson still does not authorise skipping a question. The save-and-resume point is wherever the pupil got to; there is no way to mark a question complete without completing it, and no teacher bypass.

If the internet or the devices are unavailable

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.